

Software Engineering Lab - 3

Component Modelling & Architectural Pattern Selection

Name: Kaushik Krishna Dwarakanath

SRN: PES1UG24CS900

Section: 5 – ‘B’

Project Title: Airport Lost Luggage Claim & Tracking Portal

Domain: Smart Cities, Transport & Logistics | Actors: Passenger, Baggage Service Agent

1. Architecture Selection

We chose the Layered Architecture for the Airport Lost Luggage Claim & Tracking Portal, organised into a Presentation Layer, a Business Logic Layer and a Data Layer.

2. Two Reasons for the Choice

Reason 1 – Clear separation for two distinct actors: The portal serves Passengers (submitting and tracking claims) and Baggage Service Agents (verifying claims, updating status, generating reports). A layered design lets both roles share one Presentation Layer while all claim rules, validation and role checks stay in a single Business Logic Layer, so a change to the claim workflow does not affect the UI or the database schema.

Reason 2 – Moderate, well-understood scale: The portal is a single centralised airport application with a bounded feature set (registration, claim submission, tracking, notifications, admin dashboard). Microservices would add operational complexity, inter-service latency and data-consistency problems that this scale does not justify, and a plain client-server design would mix business logic into the server with no internal structure. The layered style gives the maintainability of separated concerns without distributed-systems overhead.

3. Security Advantage

The Presentation Layer never talks to the database directly — every request passes through the Business Logic Layer, which performs authentication, role-based authorisation and input validation before any data access. This single choke point prevents SQL injection and stops a passenger from reading another passenger’s claim or luggage record, since the Data Layer exposes only the parameterised IDataAccess interface. Sensitive passenger data (contact details, flight and baggage identifiers) is therefore isolated behind two layers, and security patches can be applied in one place.

4. Performance Benefit

All layers are deployed within one process/server boundary, so calls between the UI, Claim Manager and Database are local in-process or single-hop calls rather than multiple network round-trips between services. Frequently requested operations such as “Search by Tracking ID” and “View Tracking Timeline” therefore return in well under a second, and the Business Logic Layer can cache menu-style reference data and indexed tracking lookups. Slow operations such as email/SMS alerts are handled asynchronously by the Notification Service Component, so the passenger’s claim-submission response is never blocked by them.

5. Architectural Style Comparison

Style	Pros / Cons	Suitability
Layered	Clear separation of concerns, easy to maintain and test, secure layer isolation; slight layer-traversal overhead.	Selected – best fit for a single centralised portal with two actor roles.
Microservices	Independent scaling and fault isolation; high operational complexity, network latency, data-consistency issues.	Overkill for the current scale of the portal.
Client-Server	Centralised control, simple deployment; single point of	Partly suitable but lacks internal separation of concerns.

	failure, no internal structuring of logic.	
--	--	--

6. Five Components

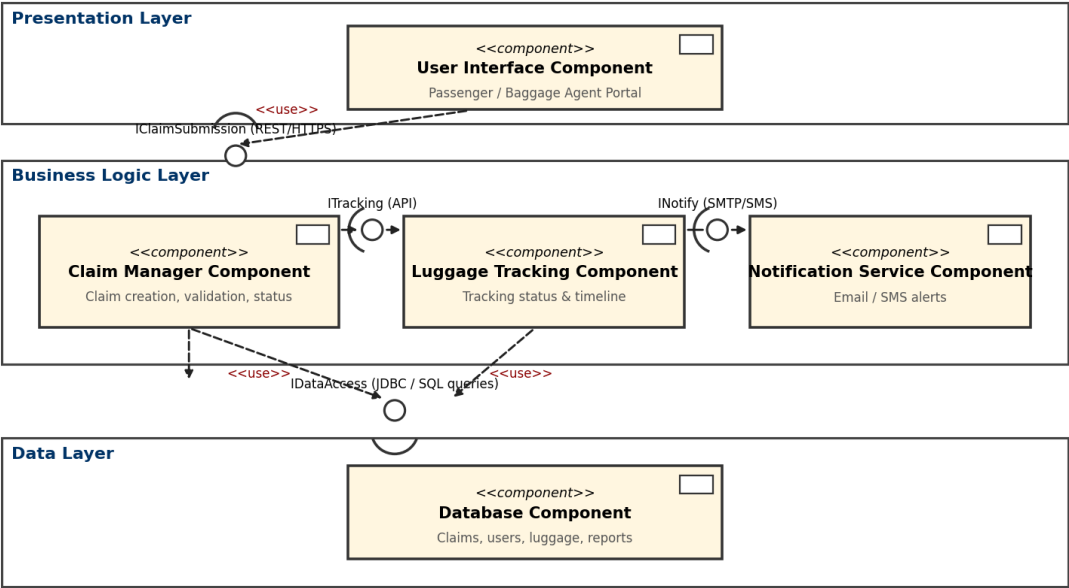
Component	Responsibility
User Interface Component	Web/touch interface for Passenger and Baggage Service Agent — claim forms, tracking screens, admin dashboard.
Claim Manager Component	Creation, editing, validation, cancellation and verification of lost-luggage claims; core business rules and authorisation.
Luggage Tracking Component	Maintains luggage status, tracking timeline and search by Tracking ID.
Notification Service Component	Sends email/SMS alerts and real-time status updates to passengers.
Database Component	Persists users, claims, supporting documents, luggage records and report data.

7. Four Required Interfaces

Interface	Between	Purpose / Technology
IClaimSubmission	User Interface → Claim Manager	Claim creation, editing and status display (REST over HTTPS).
ITracking	Claim Manager → Luggage Tracking	Tracking status and timeline queries (internal API call).
INotify	Claim Manager → Notification Service	Triggers email/SMS status alerts (SMTP / SMS gateway).
IDataAccess	Claim Manager / Tracking → Database	Data persistence and retrieval (JDBC / parameterised SQL).

8. UML Component Diagram

Airport Lost Luggage Claim & Tracking Portal – Layered Architecture (UML Component Diagram)



Interfaces: 1) UI ↔ Claim Manager 2) Claim Manager ↔ Luggage Tracking 3) Claim Manager ↔ Notification Service 4) Claim Manager ↔ Database

Ball = provided interface, Socket = required interface, dashed arrow = <<use>> dependency

Kaushik Krishna Dwarakanath | PES1UG24CS900 | Section 5-B

Notation: ball = provided interface, socket = required interface, dashed arrow with <<use>> = usage dependency.